



Tecnológico de Monterrey

Tecnológico de Monterrey, Estado de México
Inteligencia Artificial Avanzada para la ciencia de datos

Fecha de entrega: 15/09/2026

M2: momento de retroalimentación

Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución.

A01751729- Andrés Méndez Cortez

Profesor: Jorge Adolfo Ramírez Uresti

Descripción del algoritmo: Random Forest Classifier

El modelo implementado es un Random Forest Classifier (Bosque Aleatorio), un método de ensamble basado en árboles de decisión, aplicado a un problema de clasificación binaria: predecir si un cliente comprará (1) o no comprará (0) un automóvil, a partir de seis variables descriptivas del vehículo y del cliente.

Funcionamiento del algoritmo

- **Bagging (Bootstrap Aggregating):** Cada árbol se entrena con una muestra aleatoria del conjunto de entrenamiento obtenida mediante bootstrap (muestreo con reemplazo), de modo que distintos árboles ven distintos subconjuntos de observaciones.
- **Selección aleatoria de variables:** en cada división de un árbol, el algoritmo no evalúa todas las variables disponibles, sino un subconjunto aleatorio de ellas. Esto decorrelaciona los árboles entre sí y evita que unas pocas variables dominantes generen árboles muy parecidos.
- **Votación mayoritaria:** una vez entrenados los N árboles (en este caso, hasta 500 según el hiperparámetro óptimo), la predicción final se obtiene por voto mayoritario de todos los árboles del bosque. Esta agregación reduce la varianza del modelo respecto a un único árbol de decisión.
- **¿Por qué reduce el sobreajuste?:** un árbol de decisión individual tiende a memorizar particularidades del set de entrenamiento (alta varianza, sobreajuste). Al promediar las decisiones de muchos árboles poco correlacionados entre sí, el Random Forest reduce esa varianza sin incrementar sustancialmente el sesgo, logrando mejor generalización sobre datos nuevos.

Preprocesamiento de datos

Dado que el dataset original contiene variables categóricas con una relación de orden intrínseca, se optó por una codificación ordinal (OrdinalEncoder) en lugar de One-Hot Encoding, preservando así la jerarquía natural de cada categoría en vez de tratarlas como clases sin relación entre sí. La codificación aplicada fue la siguiente:

Variable	Tipo	Codificación ordinal aplicada
Precio	Categórica ordinal	Alto > Medio > Bajo
Kilometraje	Categórica ordinal	Alto > Medio > Bajo
Antigüedad	Categórica ordinal	Nuevo > Seminuevo > Usado
Marca	Categórica ordinal	Premium > Media > Económica
Historial_Accidentes	Binaria	No = 0, Sí = 1
Financiamiento_Disponible	Binaria	No = 0, Sí = 1

Pipeline y prevención de fuga de datos

El preprocesamiento (ColumnTransformer con los OrdinalEncoder) y el modelo (RandomForestClassifier) se integraron dentro de un único Pipeline de scikit-learn. Esta decisión es metodológicamente relevante: al envolver ambos pasos en un Pipeline y pasarlo completo a GridSearchCV con validación cruzada, la codificación se ajusta (fit) exclusivamente con los datos de entrenamiento de cada pliegue (fold), nunca con los datos de validación ni con el conjunto de prueba. Esto evita la fuga de información (data leakage) que ocurriría si el preprocesamiento se ajustara una sola vez sobre todo el dataset antes de dividirlo.

Optimización de hiperparámetros

Se utilizó GridSearchCV con validación cruzada de 5 particiones (cv=5) para explorar de forma sistemática 36 combinaciones de hiperparámetros ($3 \times 4 \times 3$), evaluando un total de 180 ajustes de modelo. La búsqueda se optimizó respecto al F1-Score (scoring='f1'), y no respecto a Accuracy, por las razones que se explican en la sección 3.

Hiperparámetro	Valores explorados	Valor óptimo
n_estimators (número de árboles)	100, 300, 500	500
max_depth (profundidad máxima)	8, 12, 15, None	8
min_samples_split	5, 10, 20	5

Los hiperparámetros óptimos encontrados fueron 500 árboles, profundidad máxima de 8 niveles y un mínimo de 5 muestras por división, lo que sugiere un modelo con capacidad suficiente para capturar los patrones del dataset, pero limitado en profundidad para reducir el riesgo de sobreajuste dado el tamaño moderado del conjunto de entrenamiento (160 registros).

Dataset

El modelo se entrenó y evaluó utilizando el archivo dataset_compra_coche.csv, compuesto por 200 registros y 6 variables predictoras (Precio, Kilometraje, Antigüedad, Marca, Historial_Accidentes y Financiamiento_Disponible), más la variable objetivo binaria Comprar (Sí/No).

Distribución de la variable objetivo

Antes de dividir los datos, es importante notar que el dataset presenta un desbalance moderado de clases: 135 registros (67.5%) corresponden a clientes que sí compraron, frente a 65 registros (32.5%) que no compraron. Este desbalance es la razón principal por la que, más adelante, se descarta el Accuracy como métrica principal de evaluación (ver sección 3).

División en entrenamiento y prueba

La partición de los datos se realizó con `train_test_split` utilizando `test_size=0.2`, `random_state=42` y `stratify=y`. El parámetro `stratify=y` es clave: garantiza que la proporción de clientes que compran/no compran se mantenga aproximadamente igual (67.5% / 32.5%) tanto en el conjunto de entrenamiento como en el de prueba, evitando que, por azar, el conjunto de prueba quede con una proporción de clases distinta a la real.

Conjunto	Registros	% del total	Rol
Dataset completo (dataset compra coche.csv)	200	100%	Fuente original
Entrenamiento (X_train / y_train)	160	80%	Ajuste de árboles + Grid Search (CV=5)
Prueba (X_test / y_test)	40	20%	Evaluación final, datos nunca vistos

- Conjunto de entrenamiento (X_train, y_train): 160 registros (80% del total), usados exclusivamente para ajustar los árboles del bosque y para la búsqueda de hiperparámetros mediante validación cruzada de 5 pliegues dentro de GridSearchCV. El modelo nunca ve el conjunto de prueba durante esta fase.
- Conjunto de prueba (X_test, y_test): 40 registros (20% del total, aproximadamente 27 casos 'Comprar' y 13 casos 'No comprar', respetando la proporción original gracias a la estratificación). Es una muestra completamente aislada, nunca vista por el estimador durante el entrenamiento ni la selección de hiperparámetros, y se utiliza únicamente para calcular las métricas finales reportadas en este documento.

Métricas de desempeño y justificación

Métrica principal: ¿por qué F1-Score?

La optimización del modelo (dentro de GridSearchCV) se realizó maximizando el F1-Score, y no el Accuracy, por dos razones concretas ligadas a este dataset:

- Desbalance de clases: como se mostró en la sección 2.1, el 67.5% de los registros pertenece a la clase 'Comprar'. En un dataset desbalanceado, un modelo ingenuo que siempre prediga 'Comprar' obtendría 67.5% de Accuracy sin haber aprendido ningún patrón real, lo que hace del Accuracy una métrica poco confiable como criterio único de optimización.
- Balance entre tipos de error: el F1-Score es la media armónica entre Precision y Recall. Al ser una media armónica (no aritmética), penaliza fuertemente los casos en los que una de las dos métricas es baja, obligando al modelo a mantener un equilibrio entre no generar demasiados falsos positivos (Precision) y no dejar pasar compradores reales (Recall). Esto lo

hace más robusto que optimizar por Accuracy en contextos de negocio como este, donde ambos tipos de error tienen consecuencias.

Adicionalmente, se reportan Accuracy, Recall, Precision y Specificity como métricas de contexto, para dar una lectura completa del comportamiento del modelo desde distintos ángulos.

Matriz de confusión obtenida

Sobre el conjunto de prueba (40 registros nunca vistos por el modelo) se obtuvo la siguiente matriz de confusión:

	Predicho: No comprar (0)	Predicho: Comprar (1)
Real: No comprar (0)	9 (Verdadero Negativo)	4 (Falso Positivo)
Real: Comprar (1)	0 (Falso Negativo)	27 (Verdadero Positivo)

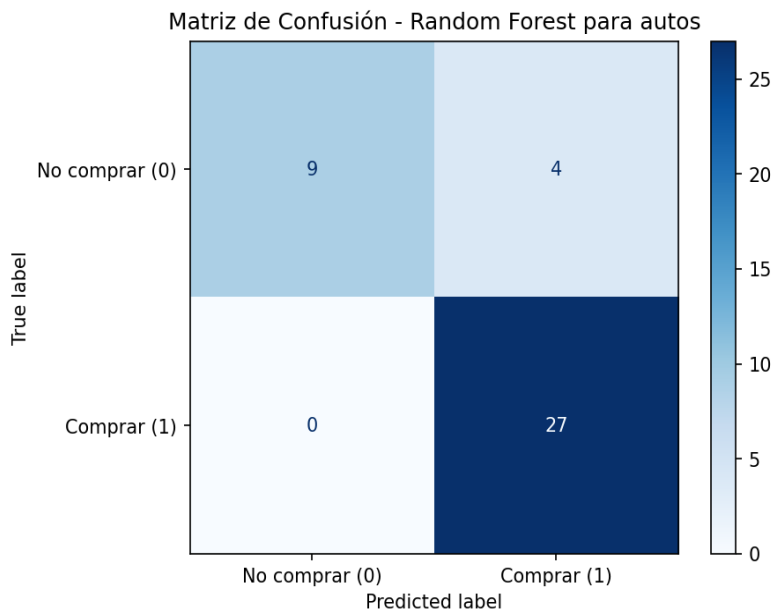


Fig. 1. Matriz de confusión del Random Forest sobre el conjunto de prueba (40 registros).

Tabla de métricas y su interpretación

Métrica	Valor	Interpretación
F1-Score	0.9310	Media armónica entre precisión y exhaustividad. Métrica de optimización elegida en GridSearchCV por el desbalance de clases (67.5% Sí / 32.5% No) y por ser la que mejor pondera falsos positivos y falsos negativos simultáneamente.
Accuracy	0.9000	El modelo acertó en el 90% de los 40 casos del conjunto de prueba. Se reporta como referencia general, pero no fue el criterio de selección por el desbalance de clases.

Métrica	Valor	Interpretación
Recall (Sensibilidad)	1.0000	De los 27 clientes que sí compraron en el conjunto de prueba, el modelo identificó a los 27 (0 Falsos Negativos). No se pierde ninguna oportunidad de venta real.
Precision	0.8710	De todos los casos que el modelo etiquetó como 'Comprar', el 87.10% efectivamente compró (27 de 31 predicciones positivas).
Specificity	0.6923	De los 13 clientes que realmente no compraron, el modelo identificó correctamente a 9 (69.23%). Es la métrica más débil, explicada por la baja cantidad de registros negativos en el set de prueba (13 casos).

El cálculo manual, a partir de la matriz de confusión (VP=27, VN=9, FP=4, FN=0), confirma cada una de las métricas reportadas:

- $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 \times (0.8710 \times 1.0000) / (0.8710 + 1.0000) \approx 0.9310$
- $\text{Accuracy} = (\text{VP} + \text{VN}) / \text{Total} = (27 + 9) / 40 = 0.9000$
- $\text{Recall} = \text{VP} / (\text{VP} + \text{FN}) = 27 / (27 + 0) = 1.0000$
- $\text{Precision} = \text{VP} / (\text{VP} + \text{FP}) = 27 / (27 + 4) \approx 0.8710$
- $\text{Specificity} = \text{VN} / (\text{VN} + \text{FP}) = 9 / (9 + 4) \approx 0.6923$

Análisis y conclusión sobre el desempeño obtenido

Prueba de que el algoritmo funciona correctamente

La evidencia de que el algoritmo funciona correctamente para este problema se sustenta en tres observaciones directas de la matriz de confusión y las métricas derivadas:

- Desempeño muy por encima de la línea base: el modelo clasificó correctamente 36 de los 40 registros del conjunto de prueba (90% de Accuracy), un porcentaje muy superior al 67.5% que obtendría un clasificador ingenuo que siempre prediga 'Comprar' (línea base o baseline por desbalance de clases). Esto confirma que el modelo efectivamente aprendió patrones útiles de las variables Precio, Kilometraje, Antigüedad, Marca, Historial_Accidentes y Financiamiento_Disponible, y no simplemente la proporción de clases.
- Recall perfecto (1.0000): con 0 Falsos Negativos sobre 27 casos positivos reales, el modelo nunca clasificó como 'No comprar' a un cliente que efectivamente compró. Esto demuestra que el árbol de decisión agregado (bosque) logró capturar el patrón completo asociado a la clase minoritaria de interés en el conjunto de prueba.
- Errores concentrados y explicables: los únicos errores del modelo (4 Falsos Positivos) se concentran en un solo tipo de confusión, el predecir 'Comprar' cuando el cliente no compró, y no se distribuyen de forma errática entre ambos tipos de error. Esto es consistente con un

modelo que aprendió una frontera de decisión razonable, ligeramente sesgada hacia la clase mayoritaria del dataset de entrenamiento (67.5% 'Sí'), en lugar de un modelo que falla de manera aleatoria.

En conjunto, estos tres puntos: desempeño superior a la línea base, ausencia total de falsos negativos y errores concentrados y explicables, constituyen evidencia de que el Random Forest implementado funciona correctamente y de forma consistente con la estructura de los datos, y no que sus resultados se deban al azar o a una fuga de datos, dado que la evaluación se realizó exclusivamente sobre el conjunto de prueba estratificado, nunca visto durante el entrenamiento ni la validación cruzada.

Overfitting vs. underfitting

La configuración óptima encontrada (500 árboles, profundidad máxima de 8, mínimo de 5 muestras por división) es una configuración moderada: suficientemente profunda para capturar relaciones no triviales entre las variables, pero acotada para no memorizar el ruido del conjunto de entrenamiento. El hecho de que la métrica de prueba ($F1 = 0.9310$) sea alta y consistente con un modelo regularizado ($\text{max_depth}=8$, en lugar de árboles sin límite de profundidad) es un indicio de que el modelo generaliza razonablemente bien y no se encuentra en un régimen de sobreajuste severo. No obstante, dado que el conjunto de prueba es pequeño (40 registros), estas métricas deben interpretarse con cautela: pequeñas variaciones en la partición de datos podrían modificar los resultados en algunos puntos porcentuales.

Conclusión

El Random Forest Classifier implementado mediante scikit-learn, dentro de un Pipeline con codificación ordinal y optimizado mediante GridSearchCV con validación cruzada de 5 pliegues, alcanzó un desempeño sobresaliente en la predicción de compra de automóviles: F1-Score de 0.9310, Accuracy de 0.90, Recall perfecto de 1.00, Precision de 0.8710 y Specificity de 0.6923 sobre un conjunto de prueba de 40 registros nunca vistos por el modelo.

El punto más relevante desde una perspectiva de negocio es el Recall perfecto: la empresa no perdería ninguna oportunidad de venta real, ya que el modelo identifica a todos los clientes dispuestos a comprar. El costo de esta configuración son 4 falsos positivos, es decir, 4 prospectos que serían contactados innecesariamente. En un entorno comercial donde el costo de perder una venta (falso negativo) es considerablemente mayor que el costo de contactar a un prospecto adicional que finalmente no compra (falso positivo), esta configuración del modelo resulta altamente conveniente y justifica la elección del F1-Score, ponderado hacia el Recall, como criterio de optimización.

Como trabajo futuro, se recomienda ampliar el tamaño del dataset, ya que actualmente cuenta con solo 200 registros, para reducir la varianza de las métricas de prueba, y explorar técnicas de balanceo de clases para verificar si la Specificity (la métrica más débil) puede mejorarse sin sacrificar el Recall perfecto obtenido.